

FIFA World Cup Analysis (1930–2014)

Chris Maher

Overview

This notebook analyzes historical FIFA World Cup data spanning from the inaugural 1930 tournament through 2014. The dataset contains two files:

- `WorldCups.csv` : one row per tournament with high-level stats (host, winner, attendance, etc.)
- `WorldCupMatches.csv` : one row per match with detailed match-level data (teams, scores, stage, etc.)

Questions

1. **Scoring Trends:** How have goals per match changed across tournaments over time?
2. **Nation Performance:** Which nations have the strongest all-time World Cup records?
3. **Attendance Over Time:** Has World Cup popularity grown consistently?
4. **Margin of Victory:** How often are matches decided by 1 goal vs. blowouts? Are knockout matches closer than group stage ones?
5. **Home Advantage:** Does hosting the World Cup give a measurable advantage?

```
In [1]: import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
```

Data Loading & Cleaning

Load and inspect both CSVs before doing any analysis.

```
In [2]: wc_df = pd.read_csv('WorldCups.csv')
matches_df = pd.read_csv('WorldCupMatches.csv')

display(wc_df.head())
display(matches_df.head())
```

	Year	Country	Winner	Runners-Up	Third	Fourth	GoalsScored	QualifiedTea
0	1930	Uruguay	Uruguay	Argentina	USA	Yugoslavia	70	
1	1934	Italy	Italy	Czechoslovakia	Germany	Austria	70	
2	1938	France	Italy	Hungary	Brazil	Sweden	84	
3	1950	Brazil	Uruguay	Brazil	Sweden	Spain	88	
4	1954	Switzerland	Germany FR	Hungary	Austria	Uruguay	140	



	Year	Datetime	Stage	Stadium	City	Home Team Name	Home Team Goals	Away Team Goals	Away Team Name	condit
0	1930.0	13 Jul 1930 - 15:00	Group 1	Pocitos	Montevideo	France	4.0	1.0	Mexico	
1	1930.0	13 Jul 1930 - 15:00	Group 4	Parque Central	Montevideo	USA	3.0	0.0	Belgium	
2	1930.0	14 Jul 1930 - 12:45	Group 2	Parque Central	Montevideo	Yugoslavia	2.0	1.0	Brazil	
3	1930.0	14 Jul 1930 - 14:50	Group 3	Pocitos	Montevideo	Romania	3.0	1.0	Peru	
4	1930.0	15 Jul 1930 - 16:00	Group 1	Parque Central	Montevideo	Argentina	1.0	0.0	France	



```
In [3]: wc_df.info()
        matches_df.info()
```

```

<class 'pandas.DataFrame'>
RangeIndex: 20 entries, 0 to 19
Data columns (total 10 columns):
#   Column                Non-Null Count  Dtype
---  -
0   Year                  20 non-null    int64
1   Country               20 non-null    str
2   Winner                20 non-null    str
3   Runners-Up            20 non-null    str
4   Third                 20 non-null    str
5   Fourth                20 non-null    str
6   GoalsScored           20 non-null    int64
7   QualifiedTeams        20 non-null    int64
8   MatchesPlayed         20 non-null    int64
9   Attendance            20 non-null    str
dtypes: int64(4), str(6)
memory usage: 2.6 KB
<class 'pandas.DataFrame'>
RangeIndex: 4572 entries, 0 to 4571
Data columns (total 20 columns):
#   Column                Non-Null Count  Dtype
---  -
0   Year                  852 non-null    float64
1   Datetime              852 non-null    str
2   Stage                 852 non-null    str
3   Stadium               852 non-null    str
4   City                  852 non-null    str
5   Home Team Name        852 non-null    str
6   Home Team Goals       852 non-null    float64
7   Away Team Goals       852 non-null    float64
8   Away Team Name        852 non-null    str
9   Win conditions        852 non-null    str
10  Attendance             850 non-null    float64
11  Half-time Home Goals  852 non-null    float64
12  Half-time Away Goals  852 non-null    float64
13  Referee               852 non-null    str
14  Assistant 1           852 non-null    str
15  Assistant 2           852 non-null    str
16  RoundID               852 non-null    float64
17  MatchID               852 non-null    float64
18  Home Team Initials    852 non-null    str
19  Away Team Initials    852 non-null    str
dtypes: float64(8), str(12)
memory usage: 841.1 KB

```

```

In [4]: wc_df.isnull().sum()
        matches_df.isnull().sum()

```

```

Out[4]: Year          3720
        Datetime      3720
        Stage         3720
        Stadium       3720
        City          3720
        Home Team Name 3720
        Home Team Goals 3720
        Away Team Goals 3720
        Away Team Name 3720
        Win conditions 3720
        Attendance     3722
        Half-time Home Goals 3720
        Half-time Away Goals 3720
        Referee        3720
        Assistant 1    3720
        Assistant 2    3720
        RoundID        3720
        MatchID        3720
        Home Team Initials 3720
        Away Team Initials 3720
        dtype: int64

```

Cleaning WorldCups (wc_df)

The values in the Attendance column loaded with '.0' at end. Removed and converted to int data type.

```

In [36]: wc_clean = wc_df.copy()
wc_clean['Attendance'] = wc_clean['Attendance'].str.replace('.', '', regex=False).a
print(wc_clean['Attendance'])

```

```

0      590549
1      363000
2      375700
3     1045246
4      768607
5      819810
6      893172
7     1563135
8     1603975
9     1865753
10    1545791
11    2109723
12    2394031
13    2516215
14    3587538
15    2785100
16    2705197
17    3359439
18    3178856
19    3386810
Name: Attendance, dtype: int64

```

Cleaning WorldCupMatches (matches_df)

The matches dataset has 4572 rows but only 852 real match records, the rest are empty rows. Drop them by filtering on `Year`. Also check for and remove duplicate rows.

```
In [6]: # drop empty rows
matches_clean = matches_df.dropna(subset=['Year']).copy()
print(matches_clean.shape)
print(matches_clean.isnull().sum())
```

```
(852, 20)
Year                0
Datetime            0
Stage              0
Stadium            0
City              0
Home Team Name      0
Home Team Goals     0
Away Team Goals     0
Away Team Name      0
Win conditions      0
Attendance          2
Half-time Home Goals 0
Half-time Away Goals 0
Referee            0
Assistant 1         0
Assistant 2         0
RoundID            0
MatchID            0
Home Team Initials  0
Away Team Initials  0
dtype: int64
```

```
In [37]: # checking for country names with leftover HTML artifacts (rn">)
# discovered during visual inspection of the data
dirty_check = matches_df[matches_df['Home Team Name'].str.contains('<">', na=False)]
print(dirty_check['Home Team Name'].unique())
```

```
<ArrowStringArray>
[ 'rn">United Arab Emirates',    'rn">Republic of Ireland',
  'rn">Trinidad and Tobago',    'rn">Serbia and Montenegro',
  'rn">Bosnia and Herzegovina']
Length: 5, dtype: str
```

```
In [38]: # remove leftover HTML style tags from country names (rn">)
matches_clean.loc[:, 'Home Team Name'] = matches_clean['Home Team Name'].str.replace('<">', '')
matches_clean.loc[:, 'Away Team Name'] = matches_clean['Away Team Name'].str.replace('<">', '')

# verify fix worked
dirty_check = matches_clean[matches_clean['Home Team Name'].str.contains('<">', na=False)]
print(dirty_check['Home Team Name'].unique()) # should be empty []
```

```
<ArrowStringArray>
[]
Length: 0, dtype: str
```

```
In [9]: # checking for duplicates
print(matches_clean.duplicated().sum())

# visually inspecting the data revealed a potential duplicate in the 2014 final
# verifying by checking if the same MatchID appears more than once
final_2014 = matches_clean[
    (matches_clean['Year'] == 2014) &
    (matches_clean['Stage'] == 'Final')
]
print(final_2014.shape)
print(final_2014[['Year', 'Stage', 'Home Team Name', 'MatchID']])
```

```
16
(2, 20)
      Year Stage Home Team Name      MatchID
828  2014.0  Final      Germany  300186501.0
851  2014.0  Final      Germany  300186501.0
```

```
In [10]: # confirmed duplicate – same MatchID appears twice, drop all duplicate rows
matches_clean = matches_clean.drop_duplicates()
print(matches_clean.shape) # should be (836, 20)

# verify the 2014 final now appears exactly once
final_2014 = matches_clean[
    (matches_clean['Year'] == 2014) &
    (matches_clean['Stage'] == 'Final')
]
print(final_2014.shape) # should be (1, 20)
```

```
(836, 20)
(1, 20)
```

Question 1: Scoring Trends

How have goals per match changed across tournaments over time?

We expect scoring rates to change over time as the game evolved — rule changes, tactical shifts, and expansion of the tournament all potentially affect how many goals are scored per match. We compute average goals per match for each tournament year.

```
In [39]: # create new column 'Total Goals' as sum of home and away team goals per match
example = matches_clean.copy()
example['Total Goals'] = example['Home Team Goals'] + example['Away Team Goals']

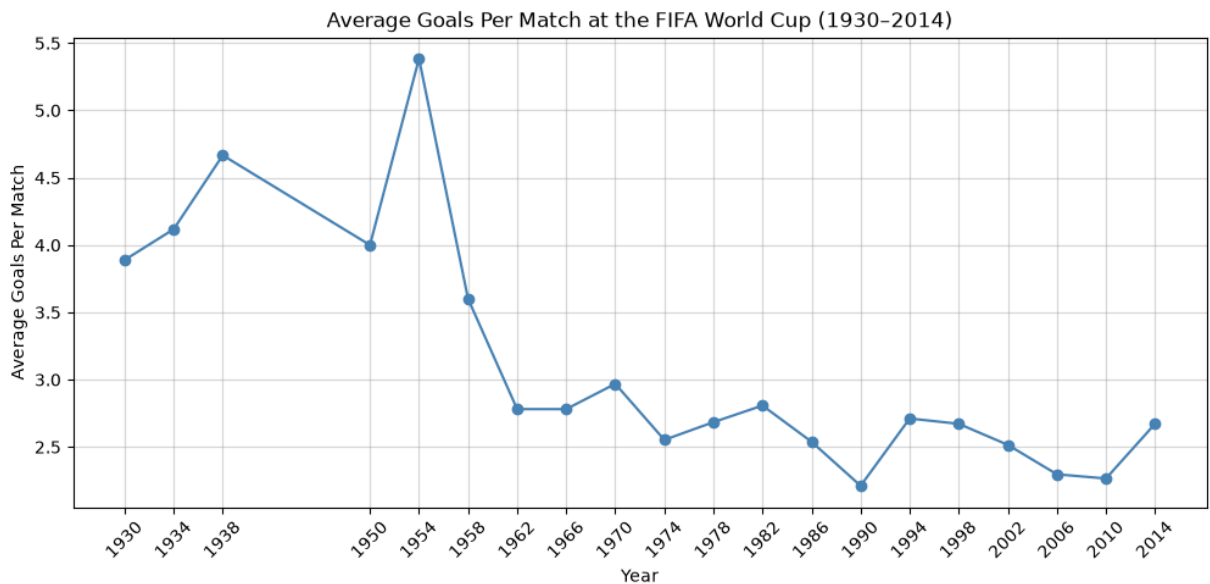
# condense to just year and total goals
new_df = example[['Year', 'Total Goals']]

# groupby year and compute average goals per match
goals_per_match = new_df.groupby('Year')['Total Goals'].mean()
```

```
goals_per_match = goals_per_match.reset_index()
goals_per_match.rename(columns={'Total Goals': 'Average Goals'}, inplace=True)
goals_per_match['Year'] = goals_per_match['Year'].astype(int)
print(goals_per_match)
```

	Year	Average Goals
0	1930	3.888889
1	1934	4.117647
2	1938	4.666667
3	1950	4.000000
4	1954	5.384615
5	1958	3.600000
6	1962	2.781250
7	1966	2.781250
8	1970	2.968750
9	1974	2.552632
10	1978	2.684211
11	1982	2.807692
12	1986	2.538462
13	1990	2.211538
14	1994	2.711538
15	1998	2.671875
16	2002	2.515625
17	2006	2.296875
18	2010	2.265625
19	2014	2.671875

```
In [12]: plt.figure(figsize=(12, 5))
plt.plot(goals_per_match['Year'], goals_per_match['Average Goals'], marker='o', col
plt.title('Average Goals Per Match at the FIFA World Cup (1930-2014)')
plt.xlabel('Year')
plt.ylabel('Average Goals Per Match')
plt.xticks(goals_per_match['Year'], rotation=45)
plt.grid(True, alpha=0.5);
```



Findings

- Early tournaments (1930s–1950s) had significantly higher scoring, peaking at **5.38 goals per match in 1954**
 - A sharp decline after 1954 brought the average down to around **2.5–2.8** where it has remained ever since
 - The modern era (1966–2014) has been remarkably consistent — the game became more defensive and tactically organized over time
 - Note: early tournaments had far fewer matches (as few as 17 in 1930 vs. 64 in modern era), so small sample sizes affect early averages
-

Question 2: Nation Performance

Which nations have the strongest all-time World Cup records?

Each match is stored as one row with a home team and an away team, so a nation's results are split across two columns. Create two separate dataframes, one from the home team's perspective and one from the away team's — then stack them to get a unified record per nation.

```
In [13]: alltime = matches_clean.copy()

# determine match result using np.select
conditions = [
    (alltime['Home Team Goals'] > alltime['Away Team Goals']),
    (alltime['Home Team Goals'] < alltime['Away Team Goals'])
]
choices = ['Home Team Win', 'Away Team Win']
alltime['Result'] = np.select(conditions, choices, default='Draw')
display(alltime.head())
```


	Year	Datetime	Stage	Stadium	City	Home Team Name	Home Team Goals	Away Team Goals	Away Team Name	condit
0	1930.0	13 Jul 1930 - 15:00	Group 1	Pocitos	Montevideo	France	4.0	1.0	Mexico	
1	1930.0	13 Jul 1930 - 15:00	Group 4	Parque Central	Montevideo	USA	3.0	0.0	Belgium	
2	1930.0	14 Jul 1930 - 12:45	Group 2	Parque Central	Montevideo	Yugoslavia	2.0	1.0	Brazil	
3	1930.0	14 Jul 1930 - 14:50	Group 3	Pocitos	Montevideo	Romania	3.0	1.0	Peru	
4	1930.0	15 Jul 1930 - 16:00	Group 1	Parque Central	Montevideo	Argentina	1.0	0.0	France	

5 rows × 21 columns



```
In [40]: # home perspective – a win for the home team is a win, a loss is when the away team
home = alltime[['Home Team Name', 'Result']].copy()
home.columns = ['Team', 'Result']
home['Win'] = (home['Result'] == 'Home Team Win').astype(int)
home['Loss'] = (home['Result'] == 'Away Team Win').astype(int)
home['Draw'] = (home['Result'] == 'Draw').astype(int)

# away perspective – reversed logic
away = alltime[['Away Team Name', 'Result']].copy()
away.columns = ['Team', 'Result']
away['Win'] = (away['Result'] == 'Away Team Win').astype(int)
away['Loss'] = (away['Result'] == 'Home Team Win').astype(int)
away['Draw'] = (away['Result'] == 'Draw').astype(int)

# stack both perspectives – us pd.concat to stack the two dataframes on top of each
# captures each country's result for each game regardless of home/away status
combined = pd.concat([home, away], ignore_index=True)
print(combined.shape)
print(combined.head(10))
```

(1672, 5)

	Team	Result	Win	Loss	Draw
0	France	Home Team Win	1	0	0
1	USA	Home Team Win	1	0	0
2	Yugoslavia	Home Team Win	1	0	0
3	Romania	Home Team Win	1	0	0
4	Argentina	Home Team Win	1	0	0
5	Chile	Home Team Win	1	0	0
6	Yugoslavia	Home Team Win	1	0	0
7	USA	Home Team Win	1	0	0
8	Uruguay	Home Team Win	1	0	0
9	Chile	Home Team Win	1	0	0

```
In [15]: # clean any remaining HTML artifacts from team names in combined
combined['Team'] = combined['Team'].str.replace(r'.">', '', regex=True)

# groupby team and sum wins, losses, draws
combined_group = combined.groupby('Team')[['Win', 'Loss', 'Draw']].sum()
display(combined_group)
```

	Win	Loss	Draw
Team			
Algeria	3	7	3
Angola	0	1	2
Argentina	42	21	14
Australia	2	8	3
Austria	12	13	4
...	...	...	...
United Arab Emirates	0	3	0
Uruguay	20	19	12
Wales	1	1	3
Yugoslavia	16	13	8
Zaire	0	3	0

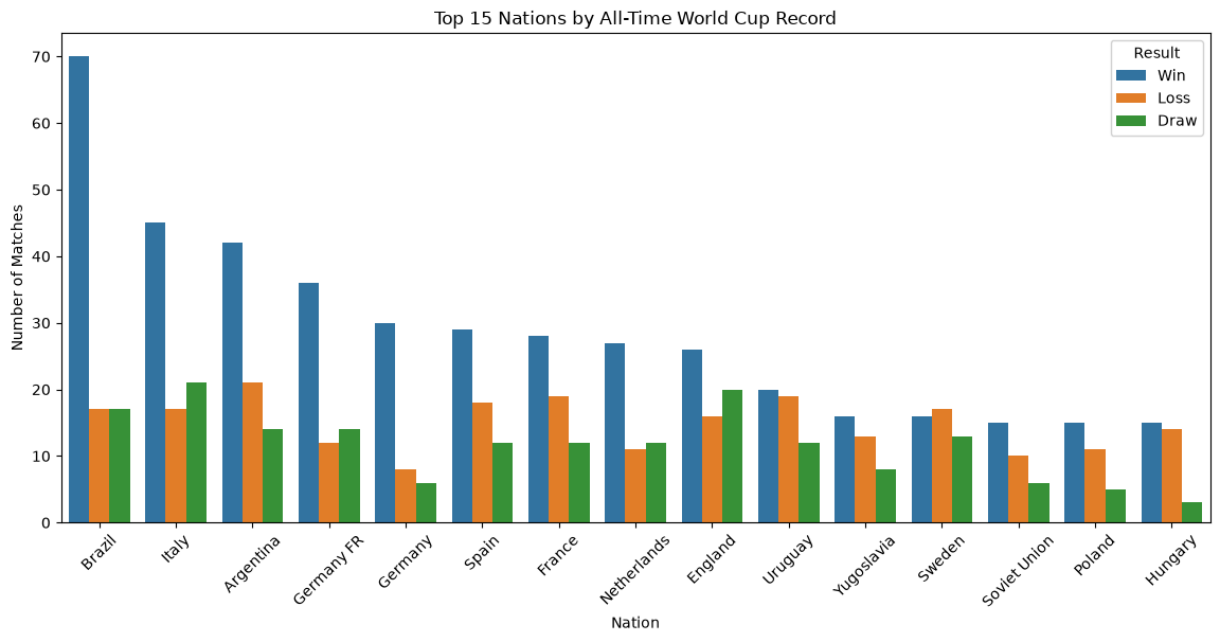
83 rows × 3 columns

```
In [41]: # sort by wins and show top 15
combined_group = combined_group.sort_values('Win', ascending=False)
top15 = combined_group.head(15).reset_index()
display(top15)
```

	Team	Win	Loss	Draw
0	Brazil	70	17	17
1	Italy	45	17	21
2	Argentina	42	21	14
3	Germany FR	36	12	14
4	Germany	30	8	6
5	Spain	29	18	12
6	France	28	19	12
7	Netherlands	27	11	12
8	England	26	16	20
9	Uruguay	20	19	12
10	Yugoslavia	16	13	8
11	Sweden	16	17	13
12	Soviet Union	15	10	6
13	Poland	15	11	5
14	Hungary	15	14	3

```
In [42]: # melt to long format for seaborn plotting
top15_melted = top15.melt(id_vars='Team', value_vars=['Win', 'Loss', 'Draw'],
                        var_name='Result', value_name='Count')

plt.figure(figsize=(14, 6))
sns.barplot(data=top15_melted, x='Team', y='Count', hue='Result')
plt.title('Top 15 Nations by All-Time World Cup Record')
plt.xlabel('Nation')
plt.ylabel('Number of Matches')
plt.xticks(rotation=45);
```



Findings

- **Brazil leads all nations with 70 wins** — a substantial gap over second-place Italy and third-place Germany
- **Germany FR and Germany** appear as separate entries because the dataset predates reunification — their records would be even stronger if combined
- The top nations show a clear pattern: high win totals come paired with relatively low loss totals, indicating consistent deep tournament runs rather than just frequent participation
- Nations like England and Yugoslavia have respectable win totals but higher loss rates, suggesting they qualify often but exit earlier

Question 3: Attendance Over Time

Has World Cup popularity grown consistently?

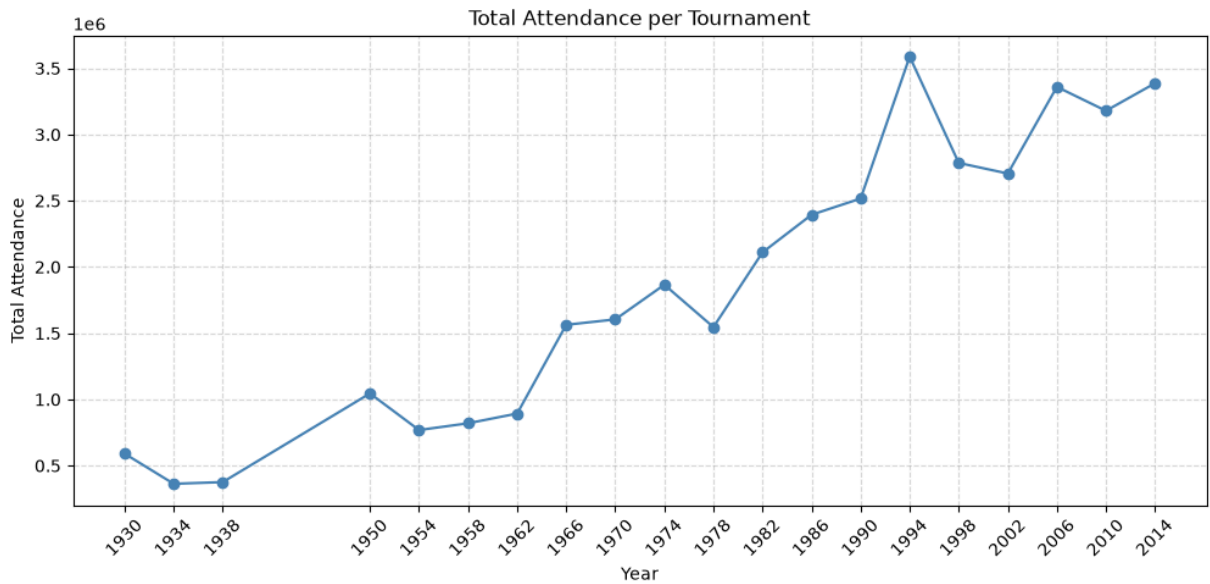
Looked at two metrics: total attendance per tournament and attendance per match. Total attendance is influenced by how many matches are played (the tournament expanded significantly over time), so normalizing by matches played gives a more honest picture of per-game popularity.

```
In [18]: bymatch = wc_clean.copy()
bymatch['Attendance per Match'] = bymatch['Attendance'] / bymatch['MatchesPlayed']
bymatch['Attendance per Match'] = bymatch['Attendance per Match'].astype(int)
display(bymatch)
```

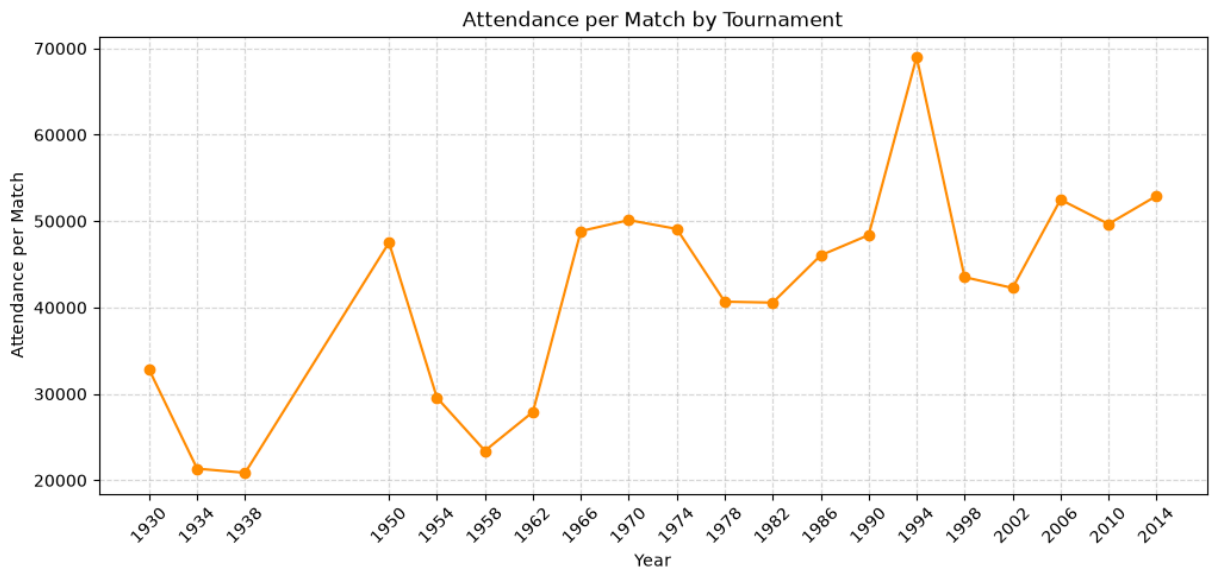
	Year	Country	Winner	Runners-Up	Third	Fourth	GoalsScored	Qual
0	1930	Uruguay	Uruguay	Argentina	USA	Yugoslavia	70	
1	1934	Italy	Italy	Czechoslovakia	Germany	Austria	70	
2	1938	France	Italy	Hungary	Brazil	Sweden	84	
3	1950	Brazil	Uruguay	Brazil	Sweden	Spain	88	
4	1954	Switzerland	Germany FR	Hungary	Austria	Uruguay	140	
5	1958	Sweden	Brazil	Sweden	France	Germany FR	126	
6	1962	Chile	Brazil	Czechoslovakia	Chile	Yugoslavia	89	
7	1966	England	England	Germany FR	Portugal	Soviet Union	89	
8	1970	Mexico	Brazil	Italy	Germany FR	Uruguay	95	
9	1974	Germany	Germany FR	Netherlands	Poland	Brazil	97	
10	1978	Argentina	Argentina	Netherlands	Brazil	Italy	102	
11	1982	Spain	Italy	Germany FR	Poland	France	146	
12	1986	Mexico	Argentina	Germany FR	France	Belgium	132	
13	1990	Italy	Germany FR	Argentina	Italy	England	115	
14	1994	USA	Brazil	Italy	Sweden	Bulgaria	141	
15	1998	France	France	Brazil	Croatia	Netherlands	171	
16	2002	Korea/Japan	Brazil	Germany	Turkey	Korea Republic	161	
17	2006	Germany	Italy	France	Germany	Portugal	147	
18	2010	South Africa	Spain	Netherlands	Germany	Uruguay	145	
19	2014	Brazil	Germany	Argentina	Netherlands	Brazil	171	



```
In [19]: plt.figure(figsize=(12, 5))
plt.plot(bymatch['Year'], bymatch['Attendance'], marker='o', color='steelblue')
plt.title('Total Attendance per Tournament')
plt.xlabel('Year')
plt.ylabel('Total Attendance')
plt.xticks(bymatch['Year'], rotation=45)
plt.grid(True, ls='--', alpha=0.5);
```



```
In [20]: plt.figure(figsize=(12, 5))
plt.plot(bymatch['Year'], bymatch['Attendance per Match'], marker='o', color='darkorange')
plt.title('Attendance per Match by Tournament')
plt.xlabel('Year')
plt.ylabel('Attendance per Match')
plt.xticks(bymatch['Year'], rotation=45)
plt.grid(True, ls='--', alpha=0.5);
```



Findings

- **Total attendance** shows a clear upward trend as the tournament expanded from 18 matches in 1930 to 64 in the modern era
- **1994 USA** is the all-time peak for both total attendance and attendance per match (~69,000 per game) — American football stadiums are significantly larger than typical soccer venues
- The fact that the US set attendance records while hosting a tournament where soccer wasn't even the most popular sport is a genuinely surprising finding

- **Attendance per match** tells a more nuanced story — dips in 1954–1962 reflect smaller European venues, and the modern era (1998–2014) has been consistently around 43,000–52,000 per match
-

Question 4: Margin of Victory

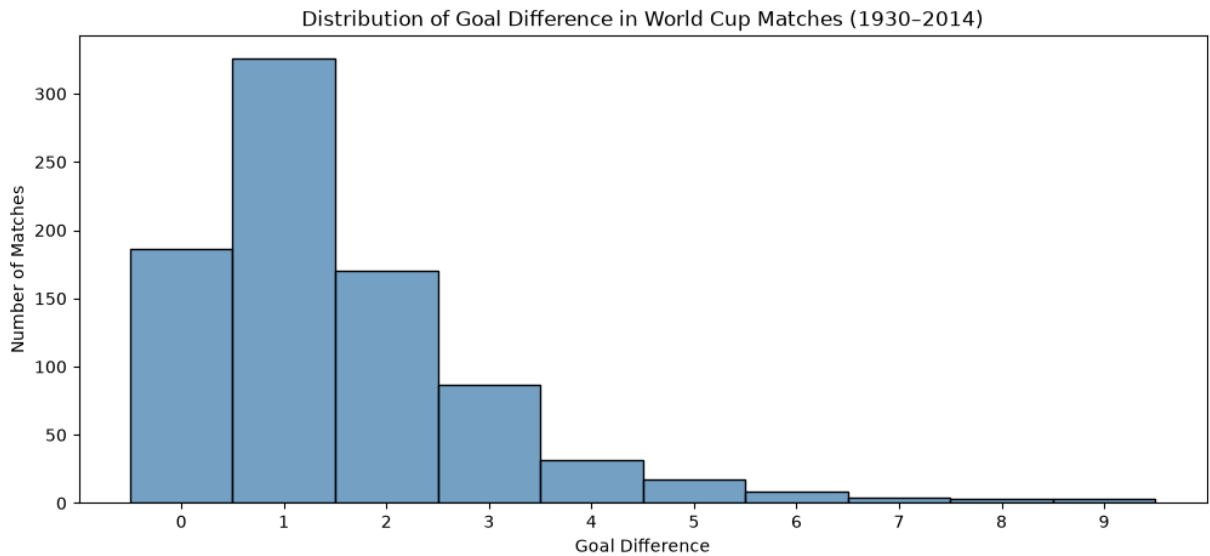
How often are matches decided by 1 goal vs. blowouts? Are knockout matches closer than group stage ones?

Compute the absolute goal difference for every match, then look at its distribution. Classify matches as group stage or knockout to compare average margins between the two.

```
In [21]: goal_diff = matches_clean.copy()
goal_diff['Goal Difference'] = (goal_diff['Home Team Goals'] - goal_diff['Away Team Goals']).abs()
print(goal_diff['Goal Difference'].value_counts().sort_index())
```

```
Goal Difference
0.0    186
1.0    326
2.0    170
3.0     87
4.0     32
5.0     17
6.0      8
7.0      4
8.0      3
9.0      3
Name: count, dtype: int64
```

```
In [22]: plt.figure(figsize=(12, 5))
sns.histplot(data=goal_diff, x='Goal Difference', bins=9, discrete=True, color='steelblue')
plt.title('Distribution of Goal Difference in World Cup Matches (1930–2014)')
plt.xlabel('Goal Difference')
plt.ylabel('Number of Matches')
plt.xticks(range(0, 10));
```



```
In [23]: # classify each match as group stage or knockout
print(goal_diff['Stage'].unique())
```

```
<ArrowStringArray>
[
    'Group 1',
    'Group 2',
    'Semi-finals',
    'Preliminary round',
    'Match for third place',
    'Group 6',
    'Group A',
    'Group C',
    'Group F',
    'Round of 16',
    'Group G',
    'Play-off for third place']
    'Group 4',
    'Group 3',
    'Final',
    'Quarter-finals',
    'First round',
    'Group B',
    'Group 5',
    'Group D',
    'Group E',
    'Group H',
    'Third place']
Length: 23, dtype: str
```

```
In [24]: group_stages = ['Group 1', 'Group 2', 'Group 3', 'Group 4', 'Group 5',
                        'Group 6', 'Group A', 'Group B', 'Group C', 'Group D',
                        'Group E', 'Group F', 'Group G', 'Group H',
                        'Preliminary round', 'First round']

conditions = [goal_diff['Stage'].isin(group_stages)]
choices = ['Group Stage']
goal_diff['Stage Type'] = np.select(conditions, choices, default='Knockout')

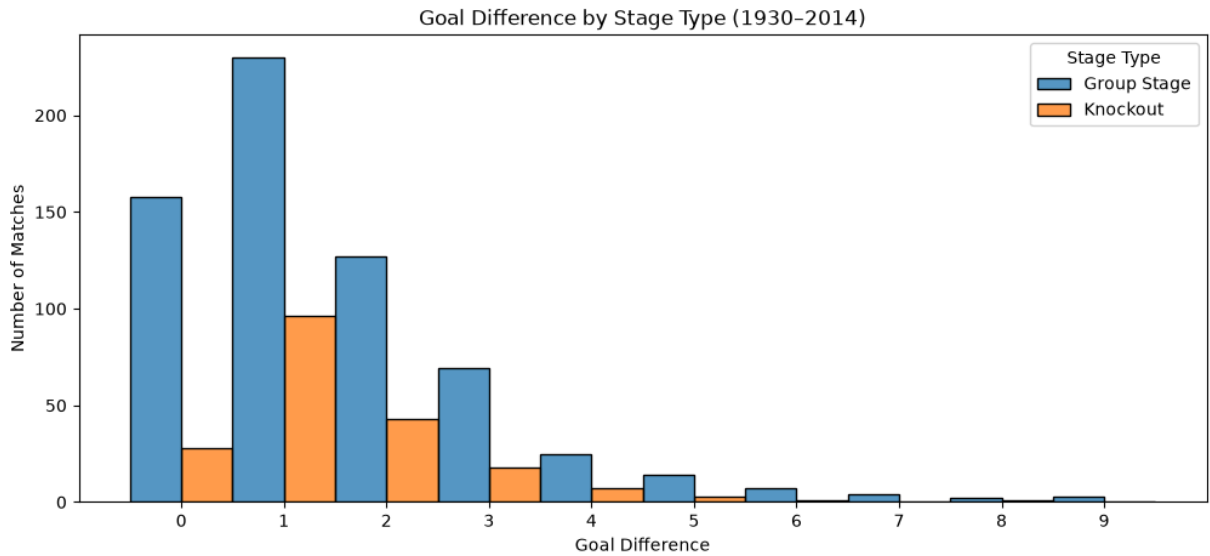
print(goal_diff['Stage Type'].value_counts())
```

```
Stage Type
Group Stage    639
Knockout       197
Name: count, dtype: int64
```

```
In [25]: # distribution by stage type
plt.figure(figsize=(12, 5))
sns.histplot(data=goal_diff, x='Goal Difference', hue='Stage Type',
             bins=9, discrete=True, multiple='dodge')
plt.title('Goal Difference by Stage Type (1930–2014)')
```



```
plt.xlabel('Goal Difference')
plt.ylabel('Number of Matches')
plt.xticks(range(0, 10));
```

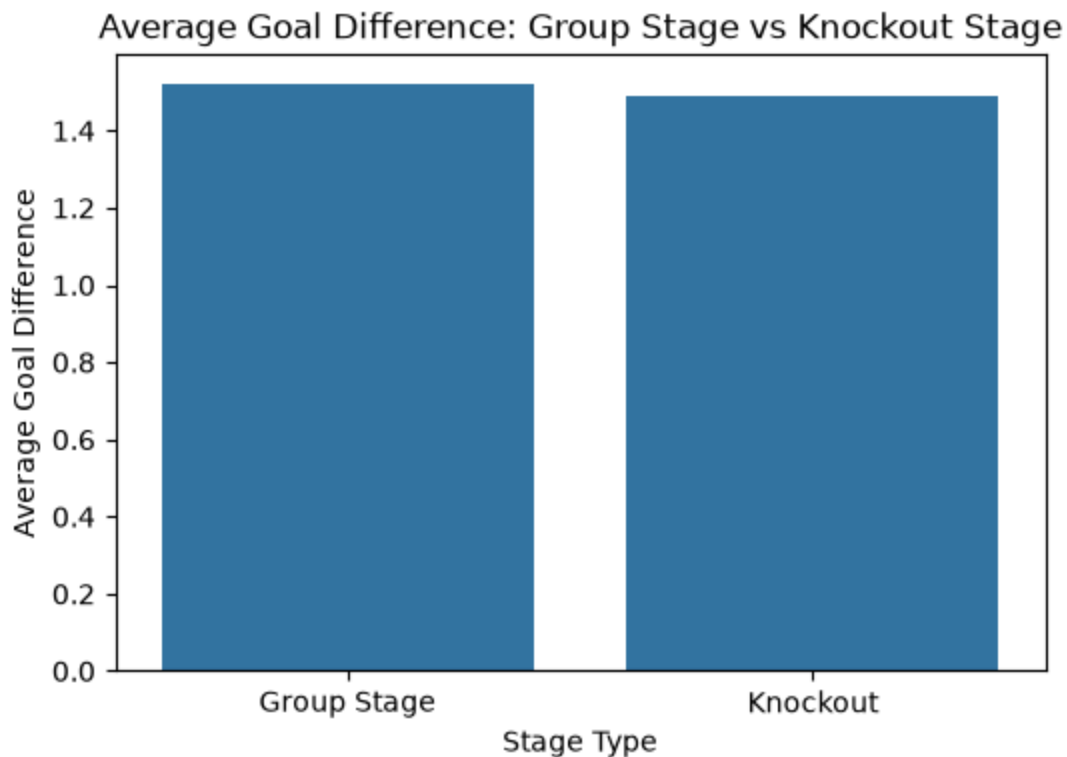


```
In [26]: # normalize by average goal difference to account for unequal match counts
print(goal_diff.groupby('Stage Type')['Goal Difference'].mean().round(2))
```

```
Stage Type
Group Stage    1.52
Knockout       1.49
Name: Goal Difference, dtype: float64
```

```
In [27]: stage_avg = goal_diff.groupby('Stage Type')['Goal Difference'].mean().round(2).reset_index()

plt.figure(figsize=(6, 4))
sns.barplot(data=stage_avg, x='Stage Type', y='Goal Difference')
plt.title('Average Goal Difference: Group Stage vs Knockout Stage')
plt.xlabel('Stage Type')
plt.ylabel('Average Goal Difference');
```



Findings

- The most common margin of victory is **1 goal**, occurring in 326 out of 836 matches (~39% of all games)
- **186 matches ended in a draw**, making draws the second most common outcome
- Together, 1-goal margins and draws account for roughly **61% of all matches** — most World Cup games are tightly contested
- Blowouts are rare — only 19 matches (less than 3%) were decided by 5 or more goals
- **Contrary to expectation, knockout matches are not meaningfully closer than group stage matches** — average goal difference is virtually identical at 1.52 (group stage) vs 1.51 (knockout)
- The high-stakes nature of knockout football does not appear to produce significantly tighter scorelines

Question 5: Home Advantage

Does hosting the World Cup give a measurable advantage?

Merge the host country for each tournament year onto the match data, identify matches where the host nation is playing, and compute their win/draw/loss rates. Compare these against an aggregate win rate for all non-host nations.

Note: The 2002 tournament was co-hosted by Korea and Japan, which required special handling — the original dataset listed the host as `Korea/Japan` which matched no team name. We split this into two separate rows so both nations are correctly identified as hosts.

```
In [28]: hosts = wc_clean[['Year', 'Country']].copy()
print(hosts)
```

	Year	Country
0	1930	Uruguay
1	1934	Italy
2	1938	France
3	1950	Brazil
4	1954	Switzerland
5	1958	Sweden
6	1962	Chile
7	1966	England
8	1970	Mexico
9	1974	Germany
10	1978	Argentina
11	1982	Spain
12	1986	Mexico
13	1990	Italy
14	1994	USA
15	1998	France
16	2002	Korea/Japan
17	2006	Germany
18	2010	South Africa
19	2014	Brazil

```
In [29]: # replace Korea/Japan with two separate rows so both nations are treated as hosts
korea_japan = pd.DataFrame({'Year': [2002, 2002], 'Country': ['Korea Republic', 'Ja
hosts_fixed = hosts[hosts['Country'] != 'Korea/Japan']
hosts_fixed = pd.concat([hosts_fixed, korea_japan], ignore_index=True)
hosts_fixed = hosts_fixed.sort_values(by=['Year'])
display(hosts_fixed)
```

	Year	Country
0	1930	Uruguay
1	1934	Italy
2	1938	France
3	1950	Brazil
4	1954	Switzerland
5	1958	Sweden
6	1962	Chile
7	1966	England
8	1970	Mexico
9	1974	Germany
10	1978	Argentina
11	1982	Spain
12	1986	Mexico
13	1990	Italy
14	1994	USA
15	1998	France
19	2002	Korea Republic
20	2002	Japan
16	2006	Germany
17	2010	South Africa
18	2014	Brazil

```
In [30]: # merge host country onto every match row by year
home_adv = matches_clean.copy()
home_adv = home_adv.merge(hosts_fixed, on='Year', how='left')
print(home_adv[['Year', 'Home Team Name', 'Away Team Name', 'Country']].head(10))
```

	Year	Home Team Name	Away Team Name	Country
0	1930.0	France	Mexico	Uruguay
1	1930.0	USA	Belgium	Uruguay
2	1930.0	Yugoslavia	Brazil	Uruguay
3	1930.0	Romania	Peru	Uruguay
4	1930.0	Argentina	France	Uruguay
5	1930.0	Chile	Mexico	Uruguay
6	1930.0	Yugoslavia	Bolivia	Uruguay
7	1930.0	USA	Paraguay	Uruguay
8	1930.0	Uruguay	Peru	Uruguay
9	1930.0	Chile	France	Uruguay

```
In [31]: # flag matches where the host nation is playing (either as home or away team)
home_adv['Host Playing'] = (
    (home_adv['Home Team Name'] == home_adv['Country']) |
    (home_adv['Away Team Name'] == home_adv['Country'])
)
print(home_adv['Host Playing'].value_counts())
```

```
Host Playing
False    794
True     106
Name: count, dtype: int64
```

```
In [32]: # filter to host matches only and determine result from host's perspective
# host could be home OR away team, so we check both directions
host_matches = home_adv[home_adv['Host Playing'] == True].copy()

conditions = [
    (host_matches['Home Team Name'] == host_matches['Country']) & (host_matches['Ho
    (host_matches['Away Team Name'] == host_matches['Country']) & (host_matches['Aw
    (host_matches['Home Team Goals'] == host_matches['Away Team Goals'])
]
choices = ['Win', 'Win', 'Draw']
host_matches['Host Result'] = np.select(conditions, choices, default='Loss')

print(host_matches['Host Result'].value_counts())
```

```
Host Result
Win      66
Draw     20
Loss     20
Name: count, dtype: int64
```

```
In [33]: # calculate host win rates
host_total = len(host_matches)
host_rates = (host_matches['Host Result'].value_counts() / host_total * 100).round(
print('Host Nation Results:')
print(host_rates)
```

```
Host Nation Results:
Host Result
Win      62.3
Draw     18.9
Loss     18.9
Name: count, dtype: float64
```

```
In [43]: # non-host comparison
# new df for all games without a host country playing
# each match has 2 non-host teams so multiply by 2 for the total appearance count
non_host = home_adv[home_adv['Host Playing'] == False].copy()
non_host_total = len(non_host) * 2
non_host_wins = (non_host['Home Team Goals'] > non_host['Away Team Goals']).sum() +
                (non_host['Away Team Goals'] > non_host['Home Team Goals']).sum()
non_host_draws = (non_host['Home Team Goals'] == non_host['Away Team Goals']).sum()
non_host_losses = non_host_total - non_host_wins - non_host_draws

print('\nNon-Host Results:')
print(f'Win: {round(non_host_wins/non_host_total*100, 1)}%')
print(f'Draw: {round(non_host_draws/non_host_total*100, 1)}%')
print(f'Loss: {round(non_host_losses/non_host_total*100, 1)}%')
```

Non-Host Results:

Win: 38.5%

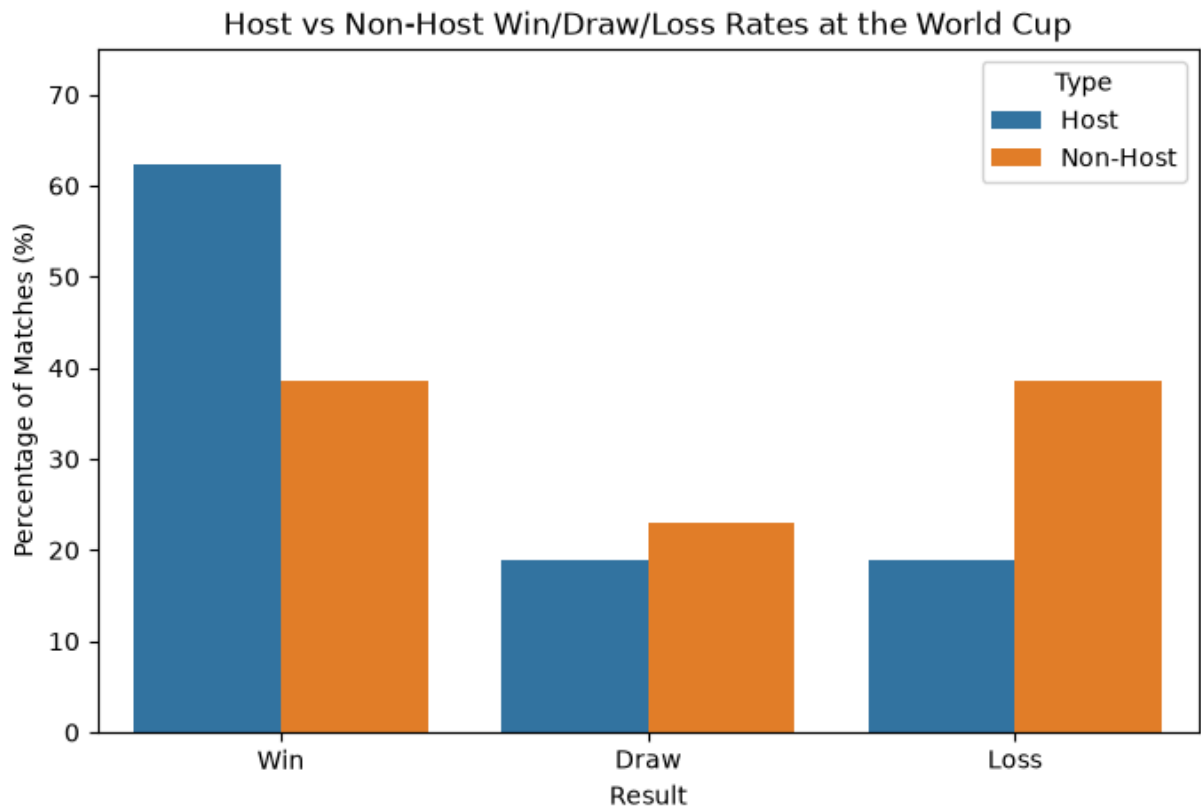
Draw: 22.9%

Loss: 38.5%

```
In [35]: # build comparison dataframe and melt for seaborn plotting
comparison_df = pd.DataFrame({
    'Result': ['Win', 'Draw', 'Loss'],
    'Host': [62.3, 18.9, 18.9],
    'Non-Host': [38.5, 22.9, 38.5]
})

comparison_melted = comparison_df.melt(id_vars='Result', var_name='Type', value_name='Percentage')

plt.figure(figsize=(8, 5))
sns.barplot(data=comparison_melted, x='Result', y='Percentage', hue='Type')
plt.title('Host vs Non-Host Win/Draw/Loss Rates at the World Cup')
plt.xlabel('Result')
plt.ylabel('Percentage of Matches (%)')
plt.ylim(0, 75);
```



Findings

- Host nations won **62.3%** of their matches compared to just **38.5%** for non-host nations — a difference of nearly 24 percentage points
- Host nations lost only **18.9%** of their matches vs. **38.5%** for non-hosts — hosts lose at less than half the rate of non-host teams
- Draw rates were similar between hosts and non-hosts (18.9% vs 22.9%), suggesting home advantage primarily converts losses into wins rather than draws
- The data strongly supports the existence of a **home advantage effect** — hosting the tournament provides a measurable and significant boost in match outcomes
- Notable caveat: the 2002 co-hosted tournament (Korea/Japan) required special handling — both nations were treated as hosts for their respective matches

Conclusion

This analysis of FIFA World Cup data from 1930–2014 produced several interesting findings:

1. **Scoring has declined dramatically** since the early tournaments — the 1954 World Cup averaged 5.38 goals per match compared to ~2.7 in the modern era, reflecting the evolution of defensive tactics
2. **Brazil is the all-time dominant nation** with 70 wins — a significant lead over Italy and Germany

3. **World Cup popularity has grown overall** but 1994 USA stands out as a massive outlier in per-match attendance, driven by the size of American sports stadiums
4. **Most matches are tightly contested** — 61% are decided by 1 goal or end in a draw. Contrary to expectation, knockout matches are not meaningfully closer than group stage matches (1.52 vs 1.51 average goal difference)
5. **Home advantage is real and significant** — host nations win 62.3% of their matches vs. 38.5% for non-host nations, a nearly 24 percentage point gap